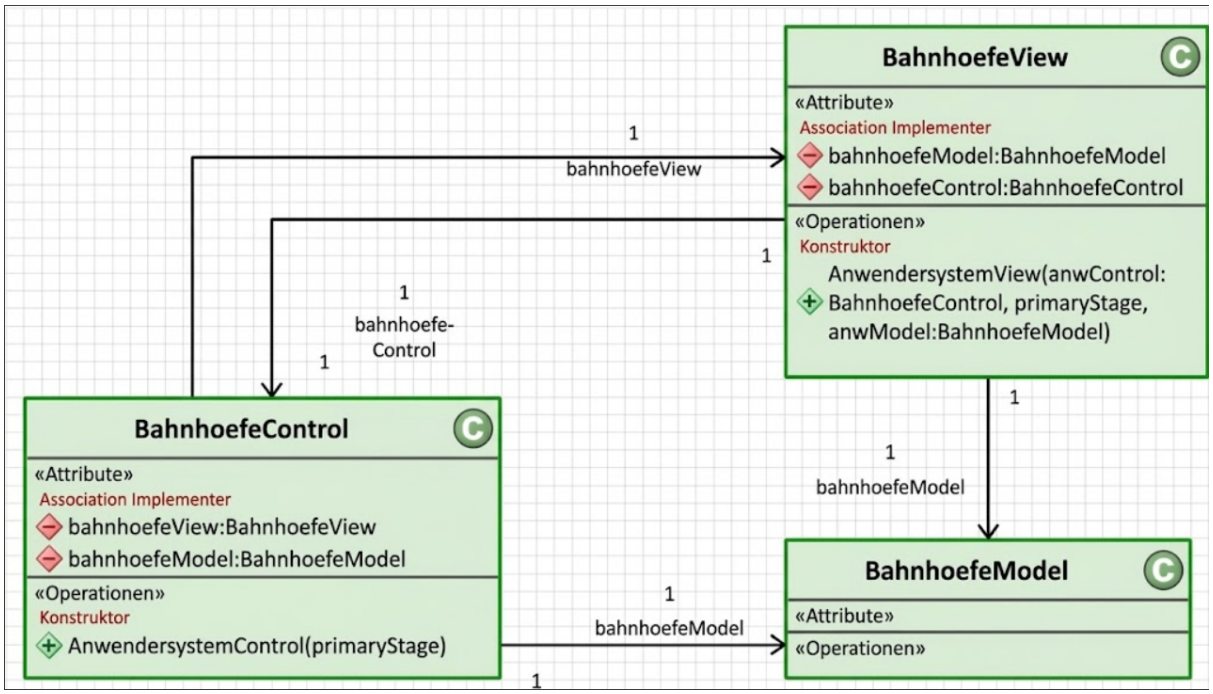
1. Aufgabe: Versionierung/Versionsverwaltung (5 Punkte)

Erläutern Sie kurz (2-3 Sätze): Was versteht man im Zusammenhang mit Versionsverwaltung unter Merge?

2. Aufgabe: Design Patterns (40 Punkte)

Gegeben ist ein UML-Klassendiagramm zum Entwurfsmuster **Model View Control (MVC)**. Dieses soll für die Struktur einer Anwendung zur Verwaltung von Bahnhöfen angewendet werden.



Gegeben sind weiterhin die folgenden Klassen `Main`, `Bahnhof` und `BahnhoefeView`.

Klasse Main

```
import javafx.application.Application;
import javafx.stage.Stage;
public class Main extends Application {
    @Override
    public void start(Stage primaryStage) {
        new BahnhoefeControl(primaryStage);
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Klasse Bahnhof

```
public class Bahnhof {
    private String name;
    private int anzahlGleise;
    private int jahrLetzteRenovierung;
    // Standardkonstruktor zur Belegung der Attribute und
    // get- und set-Methoden
    public String gibAttributeZurueck() {
        return this.getName() + ", " + this.getAnzahlGleise()
                + " Gleise, letzte Renovierung: "
                + this.getJahrLetzteRenovierung();

    }
}
```

Klasse BahnhoefeView

```
// Importe
public class BahnhoefeView {
    // Hier ergaenzen!



    private GridPane grid = new GridPane();
    private TextField txtBahnhoefe = new TextField();
    private Button btnBahnhoefe = new Button("Bahnhoefe lesen");

    public BahnhoefeView(/* Hier ergaenzen! */


    ){
        // Hier ergaenzen!
```

```
        this.grid.setAlignment(Pos.CENTER);
        this.grid.setHgap(15);
        Scene scene = new Scene(grid, 500, 150);
        primaryStage.setScene(scene);
        primaryStage.show();
        this.initKomponenten();
        this.initListener();
    }
    private void initKomponenten(){
        this.txtBahnhoefe.setPrefWidth(330);
        this.grid.add(this.txtBahnhoefe, 0, 0);
        this.grid.add(this.btnBahnhoefe, 1, 0);
    }
    private void initListener(){
        btnBahnhoefe.setOnAction(new EventHandler<ActionEvent>() {
            @Override

            public void handle(ActionEvent e) {

            }
        });
    }
    void setTxtBahnhoefe(){


    }

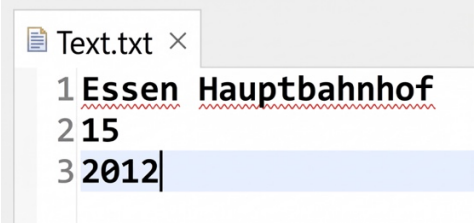
    void zeigeFehlermeldungAn(String meldung){
        Alert alert = new Alert(AlertType.ERROR);
        alert.setTitle("Fehlermeldung");
        alert.setContentText(meldung);
        alert.showAndWait();
    }
}
```

a) Geben Sie den Quellcode an, der aus dem UML-Klassendiagramm zu dem Pattern *Model View Control* hervorgeht, beziehungsweise ergänzen Sie dafür die bereits vorliegende Klasse `BahnhoefeView` an den drei gekennzeichneten Stellen, so wie Sie es in der Veranstaltung kennengelernt haben. Packages und Importe brauchen Sie nicht zu berücksichtigen. Beim Start der Anwendung soll das folgende Fenster aufgeblendet werden.



Platz für die noch fehlenden Klassen

b) Geben Sie weiterhin den Quellcode an, der ergänzt werden muss, um unter der Berücksichtigung von *Model View Control*, wie Sie es in der Veranstaltung kennengelernt haben, die folgende Funktionalität zu erhalten. Geben Sie an, in welcher Klasse und gegebenenfalls in welcher Methode der Quellcode ergänzt werden muss!
Beim Klick auf den Button mit der Aufschrift *Bahnhoefe lesen* werden die Daten zu einem Bahnhof aus der Textdatei *Text.txt* gelesen und in dem Textfeld angezeigt. Die Auswertung von Fehlerfällen soll im Control durchgeführt werden. Es soll im Fehlerfall eine Fehlermeldung erscheinen.



Hinweis: Quellcodeausschnitt zum Lesen von Daten aus der Datei:

```
BufferedReader ein
        = new BufferedReader(new FileReader("Text.txt"));

this.bahnhof = new Bahnhof(
        ein.readLine(),
        Integer.parseInt(ein.readLine()),
        Integer.parseInt(ein.readLine()));
ein.close();
```

Platz für den zu ergänzenden Quellcode

3. Aufgabe: Vertiefung Java (32 Punkte)

Gegeben ist die Klasse `Bahnhof` wie in der 2. Aufgabe. Gegeben ist weiterhin eine Klasse `BahnhoBahnhofsgebaeude` und eine Klasse `Anwendersystem`. Packages und Importe werden hier nicht berücksichtigt.

```
public class BahnhoBahnhofsgebaeude extends Bahnhof {
    private boolean mitBahnhofsgebaeude;
    public BahnhofBahnhofsgebaeude(String name, int anzahlGleise,
                                   int jahrLetzteRenovierung,
                                   boolean mitBahnhofsgebaeude) {
        super(name, anzahlGleise, jahrLetzteRenovierung);
        this.mitBahnhofsgebaeude = mitBahnhofsgebaeude;
    }
    public boolean isMitBahnhofsgebaeude() {
        return mitBahnhofsgebaeude;
    }
    public void setMitBahnhofsgebaeude(boolean mitBahnhofsgebaeude) {
        this.mitBahnhofsgebaeude = mitBahnhofsgebaeude;
    }
    @Override
    public String gibAttributeZurueck() {
        return super.gibAttributeZurueck()
                + ", mit Gebaeude: " + this.isMitBahnhofsgebaeude();

    }
}

public class Anwendersystem {
    public static void main(String[] args){
        Bahnhof bahnhofBo = new Bahnhof("Bochum Hauptbahnhof", 7, 2010);

        BahnhofBahnhofsgebaeude bahnhofEs
                = new BahnhofBahnhofsgebaeude(

                "Essen Hauptbahnhof", 15, 2012, true);

    }
}
```

a) Generics, for each

Geben Sie den Quellcode der `main`-Methode der Klasse `Anwendersystem` an, um ein Objekt vom Typ `ArrayList` unter Verwendung von Generics zu erzeugen. Die Liste soll die beiden Objekte `bahnhofBo` und `bahnhofEs`, siehe Quellcode oben, erhalten. Geben Sie den Quellcode unter Verwendung einer for each – Schleife an, so dass Sie die folgende Ausgabe erhalten. (Mit `for each` ist nicht die Verwendung einer Methode `forEach` aus gemeint!)

```
Bochum Hauptbahnhof, 7 Gleise, letzte Renovierung: 2010
Essen Hauptbahnhof, 15 Gleise, letzte Renovierung: 2012, mit Gebaeude: true
```

b) **Generics mit Typeinschränkungen**

Geben Sie den Quellcode einer Klasse `BahnhofUtil` an, welche ein Generic `T` enthält, für welches nur ein Objekt vom Typ `Bahnhof` oder einer Ableitung von `Bahnhof` eingesetzt werden kann. Es gibt ein privates Attribut `bahnhofBasis` vom Typ `T`, einen Konstruktor, welcher das Attribut mit einem Wert belegt, und eine Methode `gibBahnhofBasisAus` ohne Parameter und Rückgabewert. Diese schreibt die Attribute von `T` in die Konsole, siehe unten.
Ergänzen Sie die `main`-Methode der Klasse Anwendersystem folgendermaßen. Erstellen Sie zwei Objekte vom Typ `BahnhofUtil` unter Verwendung der Objekte `bahnhofBo` und `bahnhofEs` und erzeugen Sie die folgende Ausgabe:

```
Attribute: Bochum Hauptbahnhof, 7 Gleise, letzte Renovierung: 2010
Attribute: Essen Hauptbahnhof, 15 Gleise, letzte Renovierung: 2012, mit Gebaeude: true
```

c) **Funktionale Programmierung**

Ergänzen Sie die `main`-Methode der Klasse Anwendersystem folgendermaßen. Geben Sie den Quellcode an, um ein Objekt vom Typ `Supplier<String>` und eines vom Typ `BooleanSupplier` zu erstellen. Benutzen Sie dazu jeweils das Objekt `bahnhofEs`, geeignete Methoden aus den Klassen `Bahnhof` und `BahnhofBahnhofsgebaeude` und Lambda-Ausdrücke.
Geben Sie dann den Quellcode an, um mit Hilfe der beiden `Supplier`-Objekte die folgende Ausgabe in der Konsole zu erhalten:

```
Hat der Bahnhof Essen Hauptbahnhof ein Gebaeude? true
```

4. Aufgabe: JUnit 5 und Zustandsbasierter Test (23 Punkte)

Gegeben sind die Klassen `Bahnhof` und `BahnhofBahnhofsgebaeude` wie in der 2. Aufgabe.

JUnit 5

Ergänzen Sie den vorliegenden Quellcode eines JUnit Tests. Die Methode `setJahrLetzteRenovierung` der Klasse Bahnhof soll durch diese JUnit Testklasse getestet werden. Benutzen Sie aus der Klasse `Assertions`
`assertTrue(BooleanSupplier conditionSupplier, Supplier<String> messageSupplier)`

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;
public class BahnhofTest{
    // Ergaenzen!!!



    @BeforeEach
    public void setUp() throws Exception{
        // Ergaenzen!!!



    }

    @AfterEach
    public void tearDown() throws Exception{
        // Ergaenzen!!!



    }
    @Test
    public void test() throws Exception{
        // Ergaenzen!!!





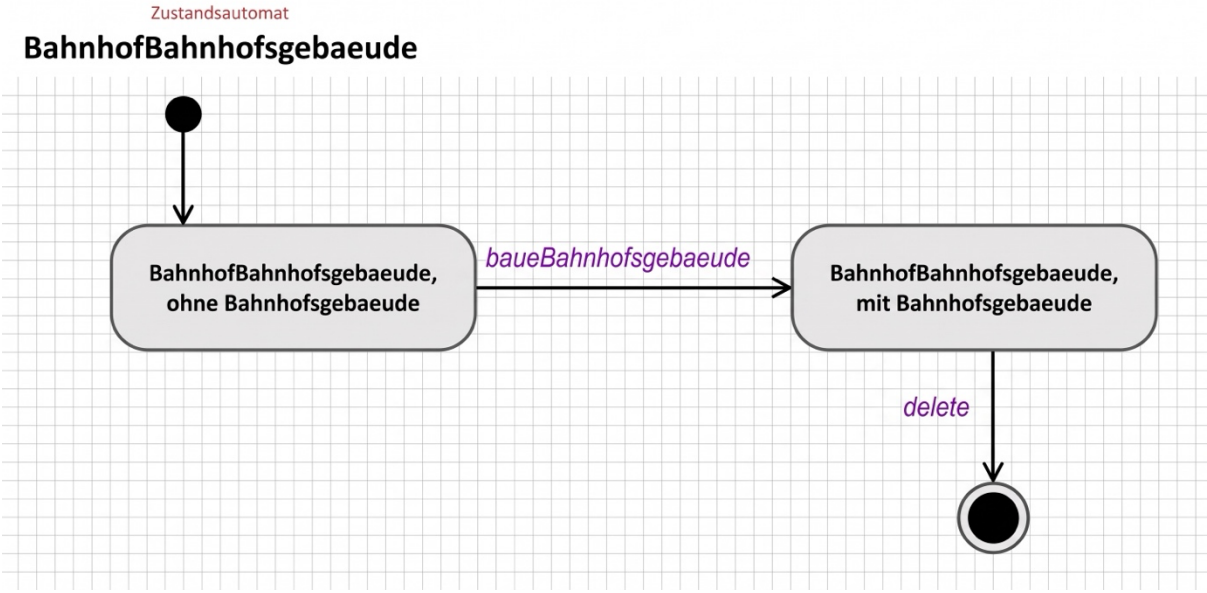




    }
}
```

Zustandsbasierter Test

Gegeben ist weiterhin ein Zustandsautomat, welcher die Zustände eines Objekts vom Typ `BahnhofBahnhofsgebaeude` modelliert.
Geben Sie den Übergangsbaum an.
Geben Sie die Testfälle inklusive Sollergebnis an, die sich aus dem Übergangsbaum ergeben.



5. Aufgabe: SOLID und Clean Code (20 Punkte)

Gegeben ist die Anwendung zu Bahnhöfen ohne und mit Bahnhofsgebäude wie in der 1. Aufgabe, der 2. Aufgabe und der 3. Aufgabe.

a) Erläutern Sie das *Interface Segregation Principle*. Nennen Sie ein Beispiel unter Verwendung der Klassen `BahnhofBahnhofsgebaeude` und `Bahnhof`.

b) Wieso trägt das Interface *Segregation Principle* zu einer höheren Wartbarkeit und Lebensdauer von Software und zu einer Reduktion der Fehleranfälligkeit bei (2-3 Sätze)?

c) Nennen und erläutern Sie kurz einen Wert aus Clean Code. Geben Sie an, wie in dem vorgegebenen Beispiel zu Bahnhöfen eine Maßnahme zur Erfüllung dieses Werts umgesetzt wurde.

d) Erläutern Sie kurz das Prinzip *Favour Composition over Inheritance (FCoI)*, welches dem 1. Grad aus Clean Code zugeordnet ist. Ist das Prinzip bereits in dem Beispiel zu Bahnhöfen umgesetzt worden? Begründen Sie kurz Ihre Antwort.